C++ Directed Graph Project Documentation

1. Overview
This project implements a directed graph with integer edge costs and a console-based user interface.
The application supports loading/saving graphs from files, graph editing operations, random graph generation, querying graph properties, and making/restoring a backup copy.

2. Project Structure
- CMakeLists.txt
  Build configuration file. Sets C++14 and defines the executable target.

- src/main.cpp
  Program entry point and console workflow.
  Contains helper functions for:
  - reading validated console input
  - readGraph(fileName) and writeGraph(fileName, graph)
  - makeRandomGraph(vertices, edges, minCost, maxCost)
  - menu rendering and command handling

- src/domain/DirectedGraph.h
  DirectedGraph class declaration.

- src/domain/DirectedGraph.cpp
  DirectedGraph class implementation.
  Stores:
  - inbound: map vertex -> vector of inbound neighbors
  - outbound: map vertex -> vector of outbound neighbors
  - costs: map (u, v) -> edge cost
  - vertices, edges counters

3. DirectedGraph API
- isEdge(u, v): bool
  Checks if edge u -> v exists.

- addEdge(u, v, cost): void
  Adds edge if missing; throws logic_error if already present.

- removeEdge(u, v): void
  Removes existing edge; throws logic_error if edge is missing.

- addVertex(): int
  Adds a new vertex and returns its identifier.

- removeVertex(vertex): void
  Removes vertex and all incident edges.

- setEdgeCost(u, v, cost): void
  Updates edge cost for an existing edge.

- getEdgeCost(u, v): int
  Returns edge cost; requires edge to exist.

- parseVertices(): vector<int>
  Returns existing vertex ids.

- parseOutboundNeighbors(vertex): vector<int>
  Returns sorted outbound neighbors.

- parseInboundNeighbors(vertex): vector<int>
  Returns sorted inbound neighbors.

- getOutDegree(vertex), getInDegree(vertex): int
  Degree queries for a vertex.

- getVertices(), getEdges(), getCosts()
  Access graph counters and edge-cost map.

4. File Format
Graph files are text files with this format:
- First line: "<number_of_vertices> <number_of_edges>"
- Next E lines: "<start_vertex> <end_vertex> <cost>"

Example:
4 5
0 1 10
0 2 4
1 3 7
2 3 1
3 0 9

5. Console Menu Operations
The menu supports:
1. Load graph from file
2. Save graph to file
3. Generate random graph
4. Show graph summary
5. List all vertices
6. Inspect a vertex (degrees + neighbors)
7. Check whether an edge exists
8. View an edge cost
9. Update an edge cost
10. Add a vertex
11. Remove a vertex
12. Add an edge
13. Remove an edge
14. Copy current graph to backup
15. Restore graph from backup copy
16. Print all edges
0. Exit

6. Validation and Error Handling
- Invalid numeric console input throws runtime_error.
- Invalid vertex ids throw logic_error.
- Invalid edge access (missing edge) throws logic_error.
- Adding an existing edge throws logic_error.
- readGraph rejects empty/invalid files and truncated edge lists.
- Random generator validates negative counts, invalid cost ranges,
  and edge counts above vertices * vertices.
- Restore backup operation fails if no backup was created yet.

7. Running the Application
From cppProject directory:
cmake -S . -B build
cmake --build build

At startup, main() tries to load graph.txt. If loading fails, the program starts with an empty graph.

8. Notes
- Self-loops are allowed because random generation and edge insertion accept u == v.
- Vertices are identified by integers.
- Vertex IDs are not reindexed after deletion; addVertex uses max(existing) + 1.